Requirements & Workload Specification Report

Delivery 1

Moreno Barragán Jhonatan David
Gaitán Castañeda Luna Valeria
Jonathan Felipe Ochoa Silva
Ortiz Luna Nicolás Francisco

October 9, 2026

Contents

1 Executive Summary **3**
1.1 Application Overview . . . 3
1.2 Key Distributed Systems Challenges . . . 3

2 System Description **4**
2.1 Business Problem . . . 4
2.2 Actors . . . 6
2.3 Main Use Cases . . . 8
2.4 System Boundaries . . . 13

3 Functional Requirements **15**
3.1 Account and Driver Management . . . 15
3.2 Ride Request and Matching . . . 15
3.3 Trip Management . . . 16
3.4 Payment and Settlement . . . 16
3.5 Feedback and Safety . . . 17

4 Non-Functional Requirements **17**
4.1 Performance and Capacity . . . 17
4.2 Availability and Reliability . . . 18
4.3 Data Management . . . 18
4.4 Geographic Scope . . . 19

5 Workload Model **19**
5.1 Workload Parameters 20
5.2 Workload Assumptions 20
5.3 Workload Scenarios 21

1 Executive Summary

1.1 Application Overview

The project consists of building a global ride-sharing platform from scratch. The system operates as a two-sided marketplace connecting riders who request transportation with drivers who provide it. The platform coordinates the complete ride lifecycle, from driver availability and ride matching to trip tracking, payment settlement, ratings, and safety support.

Table 1 summarizes the main characteristics and workload requirements of the platform.

Table 1: Platform Overview

Aspect	**Description**
System type	Two-sided, geographically distributed ride-sharing marketplace.
Main users	Riders, drivers, support and safety agents, and marketplace administrators.
Core functionality	Driver onboarding, ride requests, real-time matching and dispatch, trip lifecycle management, tracking, dynamic pricing, payment settlement, ratings, and safety/support workflows.
Average concurrency	Approximately 250,000 concurrent users.
Peak concurrency	More than 500,000 concurrent users.
Average workload	Approximately 45 million requests per hour.
Peak workload	Up to 120 million requests per hour for approximately 3–4 hours.
Availability	99.99% annual availability.
Latency	p95 response time below 500 ms for latency-sensitive operations.
Data volume	20 TB initial data volume and approximately 15 TB of annual growth.
Geographic scope	Global operation across multiple cities and regions.

1.2 Key Distributed Systems Challenges

The main challenge is coordinating a real-time marketplace in which supply and demand change continuously across geographic regions. The most relevant distributed-systems challenges are summarized in Table 2.

Table 2: Key Distributed Systems Challenges

Challenge	**System Requirement**
Real-time matching	Driver availability and rider requests must be processed with low latency because both are time-sensitive and geographically constrained.
Geospatial processing	The system must efficiently locate and evaluate nearby drivers while processing requests across many cities simultaneously.
High-frequency telemetry	Active drivers periodically send location updates, producing a continuous and write-intensive workload.
Bursty workload	The system must handle significant traffic variations caused by commuting periods, events, and localized demand surges.
Consistency	Location and availability data prioritize freshness and availability, while trip state, payments, earnings, and settlement require stronger correctness guarantees.
Fault tolerance	The platform must continue operating despite component or regional failures while protecting active trips and payment-related state.
Scalability	The system must support hundreds of thousands of concurrent users and tens of millions of requests per hour while maintaining latency requirements.
Security and safety	Driver verification, access control, ratings, fraud detection, and support workflows must operate as part of the platform.

Overall, the platform must combine low-latency real-time processing with scalability, availability, geographic distribution, and strong correctness for critical operations. These requirements establish the foundation for the functional requirements, non-functional requirements, workload model, and subsequent architectural design.

2 System Description

This section sets out what the platform has to do and the conditions it has to work under, before any architectural or technological choice is made. As the delivery requires, it stays technology-agnostic: no cloud provider, database, or implementation choice is assumed.

2.1 Business Problem

The pain point

City transportation has long carried a coordination failure between two groups that never see each other in time: people who need a ride at a given moment, and drivers with idle capacity at that same moment. The visible symptoms are long and unpredictable waits,

poor vehicle use, opaque fares, cash-only friction, and little accountability on either safety or service.

Uber's answer is to run a two-sided marketplace that connects ride demand with independent driver supply in real time, without owning the cars. The platform closes the information gap between the two sides: it finds nearby drivers, shows an up-front price and an estimated arrival time, coordinates the trip from request to drop-off, and settles payment on its own.

Who benefits

The value is different for each side of the market and for the environment around it. **Riders** get on-demand availability, predictable ETAs, transparent and cashless pricing (or cash, still common in much of Latin America), trip tracking, and safety mechanisms such as identity checks and a shareable trip status. **Drivers** get flexible earning opportunities and a demand signal that tells them where and when riders are requesting, which raises how much of their time and vehicle actually earns.

Why it is hard at scale

What makes Uber a genuine distributed-systems problem is not the raw volume on its own, but the set of constraints that have to hold at the same time for every one of those events:

1. **Local decisions on a global system.** Matching is inherently local, since it concerns drivers within a few kilometers of a rider, yet these local decisions happen at once across thousands of cities. The system has to reason at fine geographic granularity and at very large aggregate volume.
2. **Latency as a product requirement.** A rider expects a nearby driver and an ETA within seconds of requesting, and driver locations stream in constantly and must stay fresh. How fast matching responds is part of the product, not an implementation detail.
3. **Dynamic market clearing.** When demand in an area outruns supply, price has to rise to rebalance the two sides, attracting more drivers and tempering demand, and it has to do so per area and in near real time. Uber derives this surge by measuring supply and demand within hexagonal cells in each city [2].
4. **Bursty, uneven load.** Demand follows time of day, weather, and events, so peaks (rush hours, concerts, New Year's Eve) land far above the daily average and in particular places. The core matching experience has to hold up through those spikes.

5. **Different correctness needs in one request.** Parts of the platform sit at different points of the consistency–availability trade-off. Payments, driver payouts, and trip state need strong correctness and a clear audit trail; money cannot be lost or charged twice, and frequently location updates
6. **Trust and safety as requirements, not features.** The platform puts strangers together and moves real money at scale, so identity verification, ratings, fraud prevention, and trip safety are load-bearing requirements that shape the whole design.
7. **Geographic and regulatory spread.** Users sit across many countries and rule sets, which brings both latency constraints (serving users near where they are) and functional variation (pricing rules, driver requirements, and payment methods differ by market).

2.2 Actors

An actor is any entity, human or external system, that sits outside the system's boundary and interacts with it to reach a goal. Two things follow from that definition. First, parties that are affected by the platform but never interact with it directly (cities, regulators, and the wider economy from Section 2.1) are indirect stakeholders, not actors, and do not appear here. Second, components that live inside the system (the matching, pricing, or fraud logic, for example) are not actors either; only entities the system does not own qualify.

Human actors

- **Rider (Passenger) — primary.** The rider requests a trip, is matched with a driver, travels, pays, and rates the trip, this interaction is occasional but active and latency-sensitive over the request–trip–payment cycle. A rider expects a nearby driver within seconds, an accurate ETA and up-front price, a safe and trackable ride, easy payment (card or cash), and a real answer when something goes wrong.
- **Driver — primary.** The driver goes online to offer availability, receives and accepts offers, drives to and carries the rider, and gets paid. While online the interaction is continuous: location streams constantly and offers arrive in real time. A driver expects a steady flow of nearby requests, transparent and timely earnings, workable navigation, a sense of where demand is, and support when needed.
- **Customer Support / Safety Agent — secondary.** A person who handles incidents, disputes, account problems, and safety escalations, including live help during a trip. The interaction is event-driven, triggered only when a rider or driver raises something, but it has to be available around the clock. The agent needs access to trip history and live trip state, plus the tools to act (refunds, account changes, escalation).

- **Operations / Market Administrator — secondary.** An internal operator responsible for a city or region: reviewing driver onboarding, setting pricing and service-area policy, and watching marketplace health. This is administrative and periodic work, not per-trip. The administrator needs dashboards and controls, the ability to set policy per market, and visibility into supply, demand, and service quality.

System (external) actors

- **Payment Service Provider — secondary.** Authorizes and captures rider charges and pays drivers out through the supported payment methods. It is called once per trip (authorization at the start, capture at completion) plus on the payout schedule. From it the system needs secure, reliable, idempotent transactions and a firm settlement result.
- **Mapping & Geospatial Data Provider — secondary.** Supplies the base map data and geocoding used to place riders and drivers, estimate ETAs, and drive navigation. Much of the routing and geospatial logic is built in-house and sits inside the boundary; the actor here is only the external base-map and geodata source. It is used continuously and at high volume, and the system needs accurate, low-latency responses from it.
- **Identity & Background Verification Provider — secondary.** Verifies driver identity and runs background screening at onboarding, with periodic re-screening and real-time identity checks thereafter [3]. It is used mostly at onboarding and on the re-check schedule, and the system needs authoritative, auditable results from it.
- **Notification / Communications Provider — secondary.** Delivers one-time passwords, push notifications, SMS, and email (trip updates, receipts). It is used often and is event-driven, with several messages per trip, and what the system needs from it is timely, dependable delivery.

Actor catalog summary

Table 3 gathers the catalog for quick reference and for traceability in later deliveries.

Actor	Nature	Frequency	Key expectations
Rider (Passenger)	Human	Occasional, active	Fast matching, accurate ETA and price, safe trackable ride, easy payment, support
Driver	Human	Continuous while online	Steady nearby requests, transparent fair earnings, navigation, demand signal, safety
Support / Safety Agent	Human	Event-driven (24/7)	Access to trip data and tools to resolve issues and escalate safety events
Operations / Market Admin	Human	Periodic, administrative	Dashboards, per-market policy controls, marketplace visibility
Payment Service Provider	System	Per trip	Secure, reliable, idempotent transactions and settlement confirmation
Mapping & Geospatial Data	System	Continuous, high volume	Accurate, low-latency map, geocoding, and ETA data
Identity & Background Verif.	System	Onboarding + periodic	Authoritative, auditable identity and screening results
Notification / Comms	System	Event-driven, frequent	Timely, reliable delivery of messages and one-time passwords

Table 3: Actor catalog for the Uber platform (global scope).

2.3 Main Use Cases

This subsection specifies the platform's primary use cases, the ones that carry its core value. Supporting, exceptional, and purely administrative interactions are left out to keep the model focused. Each use case follows the same template: the actors involved (from the catalog in Section 2.2), preconditions, the main (success) flow, the relevant alternative flows, and postconditions. Each carries an identifier (UC-*nn*) so that functional requirements in later sections can trace back to it.

UC-01 — Request and match a ride

Primary actor

Rider (Passenger).

Secondary actors

Driver (supply side); Mapping & Geospatial Data Provider.

Preconditions

The rider is authenticated with a valid account and a registered payment method; the rider's location is available; the request channel is operational.

Main flow

1. The rider specifies pickup and destination.
2. The system estimates the route, ETA, and up-front price for the available service tiers.
3. The rider reviews the offer and confirms the request.
4. The system finds nearby available drivers and dispatches the request according to its matching policy.
5. A driver accepts the request.
6. The system confirms the match to the rider, showing driver and vehicle details and the arrival ETA.

Alternative flows

- **A1 — No supply available.** No eligible driver is found nearby; the system tells the rider and may offer to keep searching or to notify when supply appears.
- **A2 — Demand exceeds supply.** Dynamic pricing applies; the adjusted price is shown and the rider has to reconfirm before dispatch continues.
- **A3 — Offer not accepted.** The chosen driver declines or does not respond in time; the system re-dispatches to the next candidate, or times out and informs the rider.
- **A4 — Rider cancels.** The rider cancels before pickup; the system voids the pending trip and applies the cancellation policy if it applies.

Postconditions

On success, a trip is created in the "en route to pickup" state with a driver assigned, and both parties are notified. On failure, no trip is created and the rider is told why.

UC-02 — Accept and perform a trip

Primary actor

Driver.

Secondary actors

Rider; Mapping & Geospatial Data Provider.

Preconditions

The driver is onboarded, verified, and online; the driver's location is being

shared; a dispatch offer has been generated for this driver by the matching step of UC-01.

Main flow

1. The system offers a nearby trip request to the driver.
2. The driver accepts the offer.
3. The system gives navigation to the pickup point, with rider status and contact.
4. The driver reaches the pickup point and confirms pickup; the system starts the trip.
5. The system gives navigation to the destination and tracks the trip in real time.
6. The driver reaches the destination and ends the trip.

Alternative flows

- **A1 — Offer declined or ignored.** The offer expires and returns to UC-01 for re-dispatch to another driver.
- **A2 — Rider no-show.** After a set wait, the driver cancels; the no-show or cancellation policy applies.
- **A3 — Mid-trip change.** The rider asks for a stop or a new destination; the system updates route and fare accordingly.
- **A4 — Trip interrupted.** The trip ends early after an incident; the fare is adjusted and support may step in.

Postconditions

On success, the trip reaches the "completed" state awaiting settlement and hands off to UC-03. On decline or cancellation, the driver returns to the available pool.

UC-03 — Complete trip and process payment

Primary actor

Rider (the billed party); *triggered by* trip completion in UC-02.

Secondary actors

Payment Service Provider; Driver (payout beneficiary).

Preconditions

A trip has been completed; the rider has a registered payment method or an agreed cash arrangement; the fare parameters are known.

Main flow

1. On completion, the system computes the final fare (base, distance and time, any dynamic pricing, tolls and fees, and promotions).

2. The system asks the Payment Service Provider to authorize and capture the charge against the rider's payment method.
3. The provider processes the charge and returns the result.
4. The system issues a receipt to the rider and records the driver's earnings for the trip.
5. The system schedules the driver payout per the payout policy.

Alternative flows

- **A1 — Payment declined.** The system records an outstanding balance, notifies the rider, and retries or asks for another method.
- **A2 — Fare dispute.** Through UC-05, the fare is reviewed and, if warranted, corrected or refunded.
- **A3 — Cash settlement (common in Latin American markets).** The fare is paid in cash directly; the system records the cash trip and reconciles balances instead of charging a card.

Postconditions

On success, the trip is settled, the rider has a receipt, and the driver's earnings are recorded and scheduled for payout. On failure, the trip is flagged with an outstanding balance for follow-up.

UC-04 — Onboard and verify a driver

Primary actor

Driver (applicant).

Secondary actors

Identity & Background Verification Provider; Operations / Market Administrator.

Preconditions

The applicant has created an account; the market's onboarding requirements are configured.

Main flow

1. The applicant submits the required identity, license, and vehicle information and documents.
2. The system forwards identity verification and background screening to the verification provider.
3. The provider returns the identity and screening results.
4. The system, and where policy requires it an Operations Administrator, reviews the results against market requirements.

5. On approval, the system activates the driver account, letting the driver go online.

Alternative flows

- **A1 — Disqualifying result.** The application is rejected or held; the applicant is notified with the reason where permitted.
- **A2 — Incomplete documents.** The system asks for corrections and resubmission.
- **A3 — Periodic re-screening.** An active driver is re-verified on a schedule; a failure can suspend the account.

Postconditions

On approval, a verified, active driver exists and can accept trips (enabling UC-02). On rejection, no active driver account is created.

UC-05 — Rate and report a trip

Primary actor

Rider or Driver (either may rate the other).

Secondary actors

Support / Safety Agent.

Preconditions

A trip has been completed (or, for a safety report, is in progress or just ended); the participant is authenticated.

Main flow

1. After the trip, the system prompts the participant to rate the other party and, optionally, leave feedback.
2. The participant submits a rating and any optional comment or tip.
3. The system records the rating and updates the other party's aggregate score.
4. If the participant files a report (safety, service, or lost item), the system routes it to a Support / Safety Agent.
5. The agent reviews and resolves the report, taking the appropriate action per policy.

Alternative flows

- **A1 — No rating submitted.** The system moves on; the rating may be asked for again later or skipped.
- **A2 — Low rating or serious report.** The trip is flagged for review and may lead to follow-up or account action.

- **A3 — Lost item or dispute.** The case is routed to support with the relevant trip data for resolution.

Postconditions

Ratings are recorded and aggregates updated; any report carries a tracked resolution state, and quality and safety signals become available for later policy.

2.4 System Boundaries

This subsection sets the limits of the system: what sits inside its boundary (the capabilities it owns and answers for), what is explicitly out of scope, and the external interfaces through which it meets its environment. The scope is the global Uber ride-sharing platform, in line with the worldwide figures in Section 2.1; where a Latin American market differs in a way that matters, it is noted. The test throughout is one of control: a capability the platform designs, runs, and answers for is inside the boundary; a capability it depends on but does not control is outside it.

Inside the boundary (in scope)

The system owns the following:

- Rider and driver account management, profiles, and session authentication.
- Real-time supply–demand **matching and dispatch** (candidate discovery, offer, and assignment).
- **Trip lifecycle management** across its states (requested, en route to pickup, in progress, completed, settled).
- **Dynamic pricing** (surge) computation per geographic area.
- **Fare calculation and settlement orchestration**: computing fares and coordinating charges and payouts (the payment execution itself is external).
- In-house geospatial and routing logic (ETA estimation, trip tracking) built on external base-map data.
- **Ratings, feedback, and reputation** management.
- Support-case management and safety-incident routing (the workflow, not third-party services).
- Notification orchestration: deciding what to send and when (message delivery itself is external).

- Marketplace configuration and operations tooling: per-market policy, service areas, and monitoring.
- Risk and fraud logic.

LATAM note. Settlement orchestration has to handle **cash-paid trips**, which are common in several Latin American markets; for those trips the platform records and reconciles the fare rather than relying on an external card charge.

Outside the boundary (out of scope)

The following are not built or owned by the system; the platform depends on or coexists with them:

- Payment processing, banking, and settlement, handled by an external payment service provider.
- The actual identity verification and background screening (an external verification provider).
- Base map data and geocoding sources (an external mapping/geodata provider).
- Message-delivery infrastructure for SMS, push, and email (external carriers).
- Rider and driver devices, their operating systems, and network connectivity.
- The physical transportation itself, the vehicles, and their maintenance.
- Regulatory, tax, and compliance rule-making (the platform complies with rules; it does not write them).

External interfaces

Every actor in the catalog (Table 3) meets the platform across the boundary, and each crossing is an integration point. The human actors (rider, driver, support/safety agent, operations administrator) start or service interactions with the platform. The external systems are called on by the platform: the payment provider for authorization, capture, and payouts; the mapping/geodata provider for base-map and geocoding data; the identity and background verification provider for onboarding and periodic checks; and the notification provider for message delivery. These crossings are drawn in Figure 1.

System boundary diagram

Figure 1 shows the system as a single black box: it shows what the platform interacts with, not how it is built inside. Opening the box into services is left, on purpose, to the software-architecture delivery.

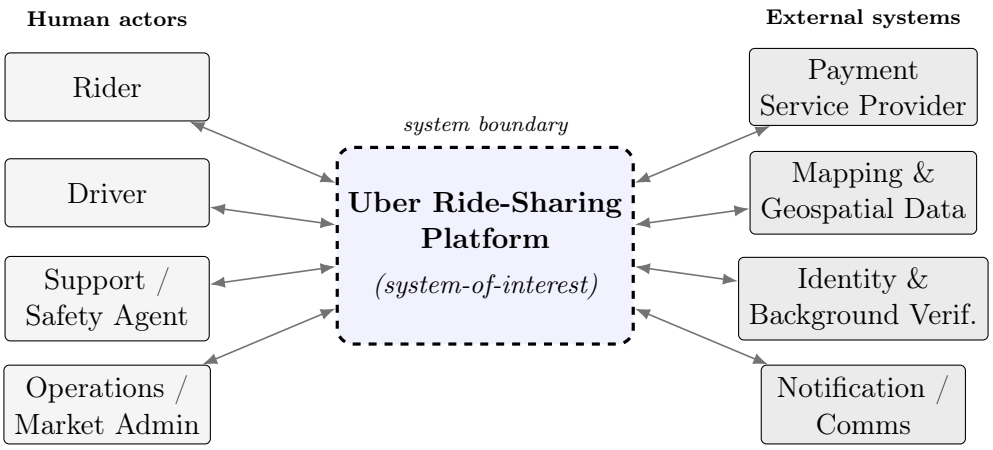


Figure 1: System boundary (context) diagram for the global Uber ride-sharing platform. The platform is shown as a black box; arrows are interactions crossing the boundary, not internal structure.

3 Functional Requirements

This section defines the main actions that system actors can perform through the ride-sharing platform. Each requirement has a unique identifier, priority, and traceability to the corresponding use case.

3.1 Account and Driver Management

ID	Actor(s)	Requirement	Priority	Use Case
FR-01	Driver Applicant	The driver applicant shall submit the required identity, license, vehicle, and verification information to complete the onboarding process.	High	UC-04
FR-02	Driver	The driver shall update their availability status and accept or reject ride assignments according to their current trip status.	High	UC-02

Table 4: Functional requirements for account and driver management

3.2 Ride Request and Matching

ID	Actor(s)	Requirement	Priority	Use Case
FR-03	Rider	The rider shall submit a ride request by providing the pickup location, destination, and selected service type.	High	UC-01
FR-04	Driver	The driver shall accept or reject a ride assignment received for an available ride request.	Critical	UC-01
FR-05	Rider	The rider shall review the estimated fare and arrival time before confirming a ride request.	High	UC-01

Table 5: Functional requirements for ride requests and matching

3.3 Trip Management

ID	Actor(s)	Requirement	Priority	Use Case
FR-06	Driver	The driver shall confirm the pickup and update the trip status during the trip until its completion or cancellation.	Critical	UC-01, UC-02
FR-07	Driver, Rider	The driver shall share their current location during an active trip, and the rider shall be able to view the trip progress.	Critical	UC-02

Table 6: Functional requirements for trip management

3.4 Payment and Settlement

ID	Actor(s)	Requirement	Priority	Use Case
FR-08	Rider	The rider shall provide or confirm a valid payment method for the completed trip and receive the corresponding payment information.	Critical	UC-03

ID	Actor(s)	Requirement	Priority	Use Case

Table 7: Functional requirements for payment and settlement

3.5 Feedback and Safety

ID	Actor(s)	Requirement	Priority	Use Case
FR-09	Rider, Driver	The rider or driver shall submit a rating and optional feedback after completing a trip.	Medium	UC-05
FR-10	Rider, Driver	The rider or driver shall report a safety incident or support issue associated with a trip.	High	UC-05

Table 8: Functional requirements for feedback and safety

4 Non-Functional Requirements

This section defines the measurable characteristics that determine how the ride-sharing platform shall behave under expected operating conditions.

4.1 Performance and Capacity

ID	Affected Actor(s)	Requirement	Target
NFR-01	Riders, Drivers	The system shall support concurrent users under average and peak operating conditions.	250,000 average; 500,000+ peak
NFR-02	Riders, Drivers	The system shall sustain the expected request workload under normal operating conditions.	45,000,000 requests/hour
NFR-03	Riders, Drivers	The system shall sustain the expected request workload during peak operating conditions.	120,000,000 requests/hour

ID	Affected Actor(s)	Requirement	Target
NFR-04	Riders, Drivers	The system shall sustain the peak workload continuously during the defined peak period.	3–4 hours
NFR-05	Riders, Drivers	The system shall maintain the required response time for latency-sensitive interactive operations under the defined workload.	p95 $<$ 500 ms

Table 9: Performance and capacity requirements

4.2 Availability and Reliability

ID	Affected Actor(s)	Requirement	Target
NFR-06	Riders, Drivers	The system shall maintain the required availability throughout its annual operating period.	99.99% annual availability

Table 10: Availability and reliability requirements

4.3 Data Management

ID	Affected Actor(s)	Requirement	Target
NFR-07	All system users	The system shall support the estimated volume of operational data required at launch.	20 TB
NFR-08	All system users	The system shall accommodate the expected annual growth of stored data while maintaining the required system functionality and performance.	15 TB/year

Table 11: Data management requirements

4.4 Geographic Scope

ID	Affected Actor(s)	Requirement	Target
NFR-09	Riders, Drivers	The system shall support ride-sharing operations across multiple geographic regions while maintaining the defined availability, capacity, and performance requirements.	Global geographic scope

Table 12: Geographic scope requirement

5 Workload Model

To accurately design the cloud-native architecture for the ride-sharing platform, we have synthesized the application's workload characteristics based on project constraints and official Uber Engineering references. The system must support over 100,000 active rides simultaneously across multiple geographies and manage high-frequency database writes.

5.1 Workload Parameters

Table 13: Workload Parameters

Parameter	Value	Justification	Reference
Average requests/hour	45,000,000	Based on continuous GPS telemetry pings from active drivers every 4 seconds.	[4]
Peak requests/hour	120,000,000	Occurs during high-demand periods triggering surge pricing.	[5]
Peak duration	3 to 4 hours	Standard morning (7-10 AM) and evening (5-8 PM) commute rushes.	[1]
Read/Write ratio	40% Read / 60% Write	Heavy write volume due to real-time geolocation updates from drivers.	Project Baseline
Availability target (SLA)	99.99%	Critical requirement allowing a maximum downtime of 52.6 minutes/year.	Project Baseline
Response time target	p95 $<$ 500 ms	Sub-second response times required for microsecond matching algorithms.	Project Baseline
Estimated data volume	20 TB	Storage for user profiles, payment tokens, and trip histories at launch.	[6]
Annual data growth	15 TB/year	Driven by trip receipts, audit trails, and historical telemetry data.	[6]
Concurrent users (avg)	250,000	Includes active riders, idle drivers, and drivers en route.	[4]
Concurrent users (peak)	500,000+	Supports the baseline of 100,000+ simultaneous active rides.	[1]
Geographic scope	Global	Requires geographic redundancy and state coordination across availability zones.	Project Baseline

5.2 Workload Assumptions

The workload model is built upon the following technical and business assumptions:

- **Telemetry Ingestion:** Every active driver app sends a geographic coordinate update via a WebSocket or streaming pipeline every 3 to 4 seconds, creating an intensive write-heavy database profile [4].

- **Matching Algorithm Execution:** The dispatch system evaluates thousands of potential rider-driver pairs using spatial indexing (Uber's H3) within microseconds [2].
- **State Management:** Distributed state across availability zones is required to handle network partitions without losing active trip data or interrupting payment coordination.

5.3 Workload Scenarios

To ensure resilience, the architecture must handle distinct operational profiles:

- **Normal Commute Scenario:** Characterized by predictable spikes between 7:00 AM - 9:00 AM and 5:00 PM - 7:00 PM. The system processes a steady stream of requests with standard dynamic pricing.
- **Extreme Surge Scenario:** A sudden, massive influx of concurrent searches and ride requests concentrated in specific urban zones. The system must instantly scale compute resources to handle surge pricing computation and real-time event streaming without degrading the sub-second response times [5].
- **Regional Outage Scenario:** If a primary availability zone experiences a failure, the geographic redundancy mechanisms must instantly route traffic to a secondary zone to preserve the 99.99% availability SLA.

References

[1] Uber Technologies, Inc., "Uber Announces Results for Fourth Quarter and Full Year 2024," *Uber Investor Relations*, Feb. 2025. [Online]. Available: https://investor.uber.com/news-events/news/press-release-details/2025/Uber-Announces-Results-for-Fourth-Quarter-and-Full-Year-2024/default.aspx (accessed Sep. 21, 2026).

[2] Uber Engineering, "H3: Uber's Hexagonal Hierarchical Spatial Index," *Uber Engineering Blog*, 2018. [Online]. Available: https://www.uber.com/us/en/blog/h3/ (accessed Sep. 21, 2026).

[3] Uber Technologies, Inc., "How Driver Screenings Work — Safety," *Uber Safety*. [Online]. Available: https://www.uber.com/us/en/ride/safety/driver-screening/ (accessed Sep. 22, 2026).

[4] Uber Engineering, "How Uber Scales Its Real-Time Infrastructure," *Uber Engineering Blog*. [Online]. Available: https://www.uber.com/en-US/blog/how-uber-scales-its-real-time-infrastructure/ (accessed Sep. 22, 2026).

[5] Uber Engineering, "The Architecture of Uber's API gateway," *Uber Engineering Blog*. [Online]. Available: https://www.uber.com/en-US/blog/architecture-api-gateway/ (accessed Sep. 22, 2026).

[6] Uber Engineering, "Uber's Big Data Platform: 100+ Petabytes with Minute Latency," *Uber Engineering Blog*. [Online]. Available: https://www.uber.com/en-US/blog/uber-big-data-platform/ (accessed Sep. 22, 2026).